7、SQL 注入漏洞与防护

一、任务 1：理解业务与查询条件

1. 简述页面的最小业务链路：

用户在页面输入查询内容，内容直接拼接到后端 SQL 查询语句中，数据库执行拼接后的 SQL，将查询结果返回页面展示。

2. 为什么输入内容能够影响数据库行为？

后端未对用户输入做隔离与过滤，将可控的用户输入直接当作 SQL 语句的一部分拼接执行，攻击者可以通过特殊字符篡改原有 SQL 逻辑，从而改变数据库查询行为。

二、任务 2：DVWA Low 级别现象观测

1. 正常输入 `1` 现象：

在 DVWA SQL Injection（Low 级别）下，输入 ID=1，页面仅返回 ID 为 1 的 admin 用户数据，业务访问正常。

2. 注入载荷 `1' OR '1'='1` 现象：

输入注入语句后，页面批量展示了数据库中所有用户（admin、Gordon、Hack、Pablo、Bob）的信息，查询条件被篡改，SQL 注入漏洞触发成功。

3. 两次结果差异说明：

程序直接拼接用户输入，未做任何过滤与转义，存在明显 SQL 注入漏洞，攻击者可通过构造恶意语句，越权获取数据库全部数据。

三、任务 3：原理分析 & 选择题

1. SQL 注入核心因果链

用户可控输入 → 直接拼接 SQL 语句 → 破坏原有查询语法结构 → 数据库非正常执行 → 数据泄露/篡改

2. 关键结论

SQL 注入的根本原因：**用户输入污染了 SQL 语句结构**，不是特殊字符本身有毒。

3. 选择题答案

1. SQL 注入的根本原因（B）

A. 特殊字符本身有害

B. 输入进入了不该控制的查询结构

C. 数据库本身不安全

2. 显错注入与盲注的主要区别（B）

A. 攻击原理完全不同

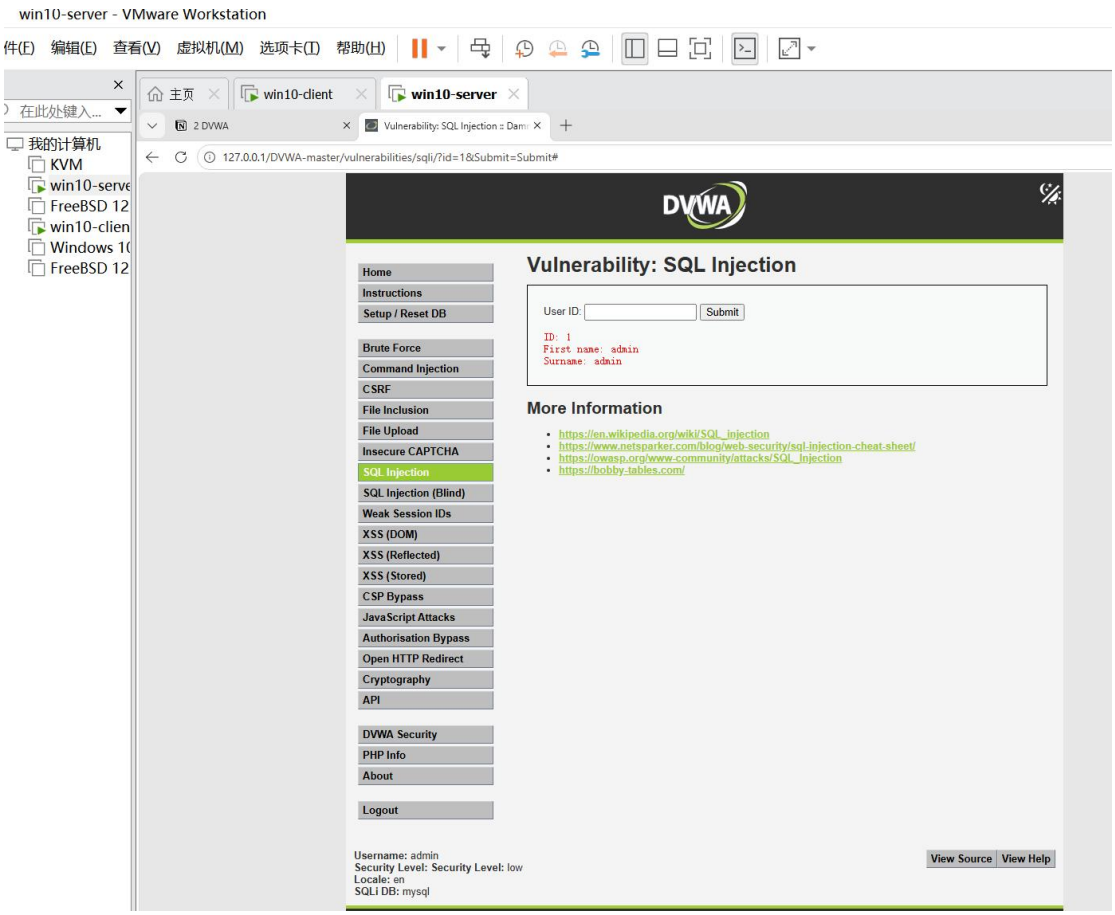
B. 漏洞证据的呈现形式不同

C. 只有盲注才有危害

4. Low 与 Medium 级别对比
- Low：无任何过滤、转义，直接拼接用户输入，可直接通过单引号篡改 SQL 逻辑，注入极其简单。
- Medium：对单引号等特殊字符进行转义过滤，可阻挡基础字符型注入，但仍存在数字型注入风险。

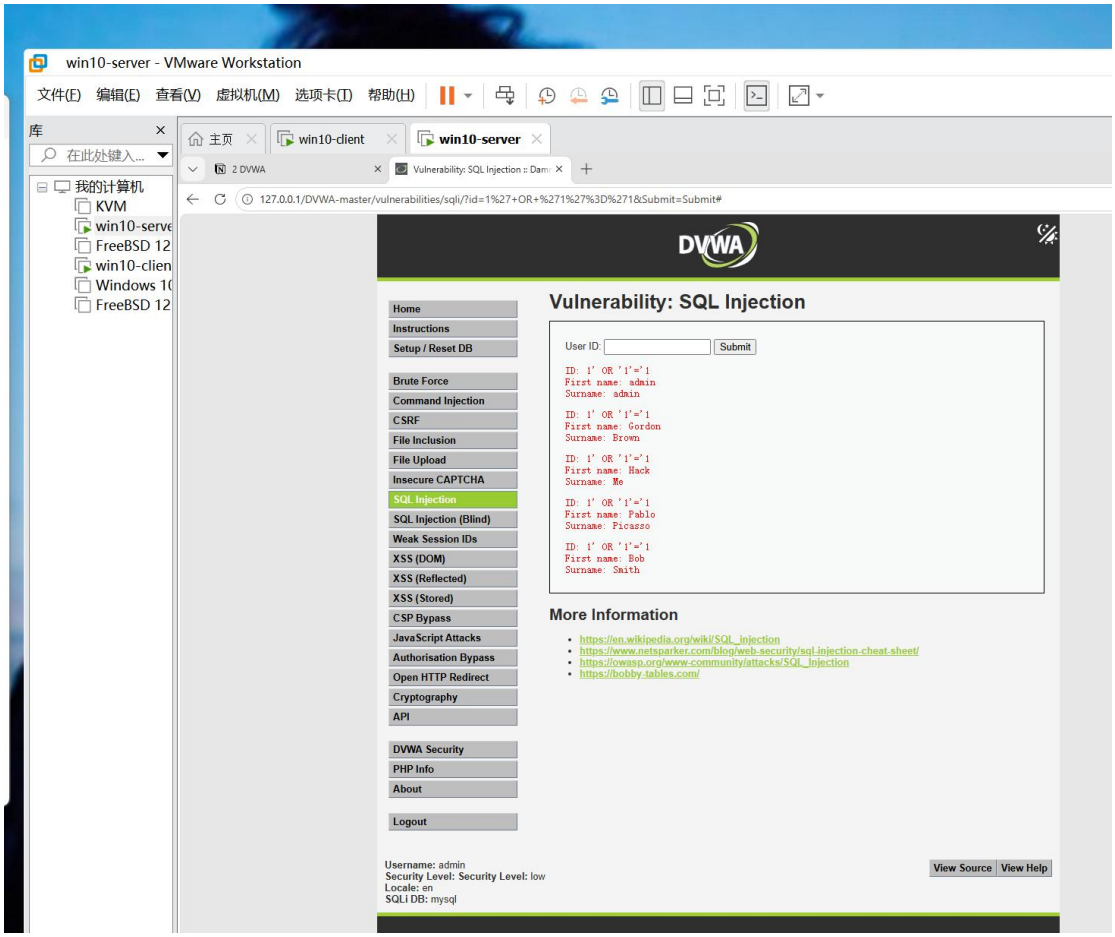
四、任务 4：SQL 注入核心防护措施（四条）
1. **使用预编译/参数化查询（最核心）**
将 SQL 语句结构与用户输入数据完全分离，杜绝语句拼接，从根源上防止结构被污染。
2. **严格输入过滤与白名单校验**
对传入参数做类型、格式、内容校验，采用白名单机制限制合法输入，阻断恶意字符注入。
3. **关闭数据库详细错误回显**
统一错误页面，避免通过报错获取库名、表名、字段等敏感信息，减少攻击线索。
4. **数据库账户最小权限部署**
Web 业务数据库账号仅分配必要查询/操作权限，降低漏洞利用后的危害范围，防止数据被批量泄露或篡改。

五、实验截图说明
1. 正常查询截图（对照组）
DVWA SQL Injection（Low 级别），输入 ID=1，仅查询出单条指定用户数据，验证业务功能正常。



2. 注入攻击成功截图（实验组）

DVWA SQL Injection（Low 级别），输入`1' OR '1'='1`，批量返回所有用户信息，证明 SQL 注入漏洞复现成功。



六、学生自评

通过本次实验，我理解了 SQL 注入的产生原理，明白了查询结构被污染是注入的核心根源，掌握了基础注入测试方法与工业级防护手段，提升了 Web 安全漏洞认知与防御意识。